

ADAR'S NUMPY NINJA GUIDE

NUMPY

COMPLETE MASTERY HANDBOOK

Arrays · Broadcasting · Linear Algebra · Vectorization

```
np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Compiled & Structured for Exam-Ready Mastery

Adar Dey · Algorithm Learner and AI Enthusiastic

PART 01 – 05 ✦ THEME: MIDNIGHT INDIGO & GOLD

Table of Contents

Part 01 — Introduction to NumPy Arrays

- 1.1 Why NumPy?
- 1.2 Python List — Heterogeneous Storage
- 1.3 NumPy Array — Homogeneous Structure & Coercion Rules
- 1.4 Python List vs. NumPy Array — Comprehensive Comparison
- 1.5 Vectors — One-Dimensional (1D) Arrays
- 1.6 Matrices — Two-Dimensional (2D) Arrays
- 1.7 Initialization Routines & Built-in Data Generators
- 1.8 Random Distribution Module Mechanics (np.random)
- 1.9 Structural Attributes, Statistical Reductions & Reshaping
- 1.10 Indexing & Slicing — 1D Vectors
- 1.11 Advanced Multi-Dimensional Indexing — 2D Matrices
- 1.12 Boolean Indexing — Conditional Data Filtering

Part 02 — Advanced Array Operations & Mechanics

- 2.1 Element-wise Arithmetic Operations
- 2.2 NumPy Broadcasting Mechanics
- 2.3 Memory Management — Views, References & Deep Copies
- 2.4 Matrix Multiplication & Transposition
- 2.5 Structural Combinations — Stacking Mechanics
- 2.6 Structural Segregation — Array Splitting Mechanics

Part 03 — Advanced Data Cleaning & Logical Pipelines

- 3.1 The Imperfect Data Toolkit — Managing NaN and Inf
- 3.2 Vectorized Conditionals via np.where()
- 3.3 Advanced Multi-Criteria Conditional Masking
- 3.4 Global Identification & Truth-Value Pipelines
- 3.5 Data Reordering — Sort vs. Argsort

Part 04 — Advanced Mathematical Tools & Benchmarking

- 4.1 Universal Functions (ufuncs) for Scaled Trig & Math
- 4.2 Elementary Linear Algebra Engine (np.linalg)
- 4.3 Collapsing Dimensions — Flatten vs. Ravel
- 4.4 Performance Benchmarking — Python Loops vs. NumPy Vectors

Part 05 — Structural Data Mutations & Vector Expansion

- 5.1 Precision Control — Explicit Type Casting (.astype)
- 5.2 Dimensional Inflation — np.newaxis & np.expand_dims
- 5.3 Dimensional Stripping — Compressing Axes (.squeeze)
- 5.4 Multi-Dimensional Coordinate Extraction (np.argwhere)
- 5.5 Data Stabilization & Value Clamping (np.clip)

PART 01

Introduction to NumPy Arrays

Foundations, Vectors, Matrices & Core Mechanics

12 Core Topics · Fully Worked Examples & Outputs

PART 1 • TOPIC 1

Why NumPy?

NumPy stands for **Numerical Python**. It is the structural bedrock of almost every scientific computing library in the Python ecosystem.

Key advantages:

- ✦ Faster than normal Python lists due to contiguous C-level memory storage.
- ✦ Uses significantly less memory than equivalent Python lists.
- ✦ Supports native, optimized vector operations without explicit loops.
- ✦ Serves as the structural bedrock for Machine Learning, Data Science, Artificial Intelligence, Deep Learning, and Computer Vision frameworks.

• • • PYTHON

```
pip install numpy

import numpy as np  # 'np' is the universal community alias
```

★ KEY INSIGHT

Every scientific Python library — pandas, scikit-learn, TensorFlow, PyTorch — is ultimately built on top of NumPy arrays.

PART 1 • TOPIC 2

Python List — Heterogeneous Storage

A standard Python list is a generic container that can store completely different data types together in the same structure. This flexibility comes with hidden overhead: every element is a pointer to a separate Python object scattered in memory.

• • • PYTHON

```
a_list = [1, 2, 3.5, "adar"]

print(a_list)
print(type(a_list[0]), type(a_list[1]), type(a_list[2]), type(a_list[3]))
```

► OUTPUT

```
[1, 2, 3.5, 'adar']

<class 'int'> <class 'int'> <class 'float'> <class 'str'>
```

PART 1 • TOPIC 3

NumPy Array — Homogeneous Structure & Coercion Rules

The Golden Homogeneity Rule: a NumPy array stores only ONE uniform data type. If mixed types are passed to `np.array()`, NumPy automatically performs type coercion (upcasting) so that every element shares one compatible dtype.

• • • PYTHON

```
a_array = np.array([1, 2, 3.5, "adar"])

print(a_array)
print(type(a_array[0]), type(a_array[1]), type(a_array[2]), type(a_array[3]))
```

► OUTPUT

```
['1' '2' '3.5' 'adar']

<class 'numpy.str_'> <class 'numpy.str_'> <class 'numpy.str_'> <class 'numpy.str_'>
```

★ KEY INSIGHT

Because one element is an explicit string, NumPy converts every numeric element into a uniform string type to preserve the contiguous-memory layout that makes arrays fast.

PART 1 • TOPIC 4

Python List vs. NumPy Array — Comprehensive Comparison

A side-by-side comparison of the two fundamental container types:

Python List	NumPy Array
◆ Can store multiple unrelated data types together. ◆ Slower execution due to scattered object pointers. ◆ Higher total memory consumption footprint. ◆ General-purpose data collection container.	◆ Stores exactly one uniform structural data type. ◆ Highly optimized, vectorized processing speed. ◆ Significantly lower memory usage footprint. ◆ Purpose-engineered for numerical computation.

PART 1 • TOPIC 5

Vectors — One-Dimensional (1D) Arrays

A **vector** is a one-dimensional array structure, mathematically written as $\text{Vector} = (x_1, x_2, x_3, \dots, x_n)$.

In Machine Learning, feature maps, weights, and input variables are standardly represented using 1D NumPy arrays.

• • • PYTHON

```
vector = np.array([10, 20, 30])
print(vector)
```

► OUTPUT

```
[10 20 30]
```

★ KEY INSIGHT

Dimension count: 1D | Shape tuple: (n,) → example (3,)

PART 1 • TOPIC 6

Matrices — Two-Dimensional (2D) Arrays

A **matrix** is a two-dimensional array arranged into explicit Rows x Columns. A 3x3 grid has a shape of (3, 3) — 3 rows and 3 columns.

• • • PYTHON

```
matrix_raw_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
matrix_numpy = np.array(matrix_raw_list)
print(matrix_numpy)
```

► OUTPUT

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

★ KEY INSIGHT

Dimension count: 2D | Shape tuple: (rows, columns) → example (3, 3)

PART 1 • TOPIC 7

Initialization Routines & Built-in Data Generators

NumPy provides several built-in generator functions to instantly create arrays without manually typing every value.

• • • PYTHON

```
# A. np.arange(start, stop, step) — stop is exclusive
np.arange(1, 11, 2)      # [1 3 5 7 9]

# B. 1D placeholders
np.zeros(5)              # [0. 0. 0. 0. 0.]
np.ones(4)               # [1. 1. 1. 1.]

# C. 2D placeholders (tuple dimensions)
np.zeros((3, 4))         # 3x4 grid of zeros
np.ones((4, 3))          # 4x3 grid of ones

# D. np.linspace(start, stop, samples) — evenly spaced values
np.linspace(0, 1, 10)
```

► OUTPUT

```
[1 3 5 7 9]

[ [0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.] ]

[ [1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.] ]

[0.      0.11111111 0.22222222 0.33333333 0.44444444
 0.55555556 0.66666667 0.77777778 0.88888889 1.      ]
```

PART 1 • TOPIC 8

Random Distribution Module Mechanics (np.random)

NumPy's random module generates values from several distributions:

• • • PYTHON

```
np.random.rand(10)       # Uniform [0.0, 1.0)
np.random.randn(10)      # Standard Normal (mean=0, std=1)
np.random.randint(10, 20) # Single random integer scalar
np.random.randint(10, 20, 10) # Array of random integers
```

► OUTPUT

```
Uniform: [0.65 0.05 0.46 0.63 0.29 0.71 0.16 0.26 0.74 0.95]
Normal: [-1.55 0.18 -0.27 0.36 -1.57 0.64 -1.08 1.97 0.29 -1.31]
Single Integer: 15
Integer Array: [14 11 14 12 14 12 14 19 13 11]
```

★ KEY INSIGHT

`rand()` is uniform; `randn()` is Gaussian/normal — the two are not interchangeable for statistical simulations.

PART 1 • TOPIC 9

Structural Attributes, Statistical Reductions & Reshaping

Every NumPy array exposes structural attributes and built-in statistical reduction methods.

• • • PYTHON

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

arr.shape    # (3, 3)
arr.size     # 9
arr.dtype    # int64
arr.min()    # 1
arr.max()    # 9
arr.sum()    # 45
arr.mean()   # 5.0
arr.std()    # 2.5819...
arr.argmax() # 8 (flattened index)
arr.argmin() # 0 (flattened index)

np.sum(arr, axis=1) # row-wise sum → [ 6 15 24]
np.sum(arr, axis=0) # column-wise sum → [12 15 18]

flat = np.arange(5, 35)
grid = flat.reshape(6, 5) # 30 items → 6 rows x 5 cols
```

► OUTPUT

```
Reshaped (6x5):
[[ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]
 [25 26 27 28 29]
 [30 31 32 33 34]]
```

★ KEY INSIGHT

`axis=1` collapses across columns (row-wise result); `axis=0` collapses across rows (column-wise result). The total element count must stay identical before and after reshape.

PART 1 • TOPIC 10

Indexing & Slicing — 1D Vectors

Standard Python slice syntax [start:stop:step] applies directly to NumPy vectors.

• • • PYTHON

```
arr = np.arange(11, 21) # [11 12 13 14 15 16 17 18 19 20]

arr[3:8] # index 3 to 7    → [14 15 16 17 18]
arr[:5]  # start to index 4 → [11 12 13 14 15]
arr[3:]  # index 3 to end   → [14 15 16 17 18 19 20]
arr[3::2] # index 3 to end, step 2 → [14 16 18 20]
```

PART 1 • TOPIC 11

Advanced Multi-Dimensional Indexing — 2D Matrices

2D indexing uses the [row, column] coordinate form, plus independent slice ranges per axis.

• • • PYTHON

```
m = np.arange(1, 31).reshape(6, 5)

m[5]      # extract row 5      → [26 27 28 29 30]
m[2, 3]    # single coordinate → 14
m[0:2, 1:3] # sub-matrix window → [[2 3],[7 8]]
m[3:, 3:]   # bottom-right corner → [[19 20],[24 25],[29 30]]
m[:, 2]     # entire column 2    → [3 8 13 18 23 28]
```

PART 1 • TOPIC 12

Boolean Indexing — Conditional Data Filtering

A boolean mask is a same-shaped array of True/False values generated from a condition. Passing the mask back into the array's brackets extracts only the matching elements.

• • • PYTHON

```
arr = np.arange(11, 21)

mask = arr % 2 == 0
print(mask) # [False True False True ...]

print(arr[mask]) # [12 14 16 18 20]
```

► OUTPUT

```
[12 14 16 18 20]
```

PART 02

Advanced Array Operations & Mechanics

Arithmetic, Broadcasting, Memory, Multiplication & Stacking

6 Core Topics · Fully Worked Examples & Outputs

PART 2 • TOPIC 1

Element-wise Arithmetic Operations

Arithmetic operators apply automatically at matching index positions between two arrays of the same shape — no manual looping required.

• • • PYTHON

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([6, 7, 8, 9, 10])

a + b  # [ 7  9 11 13 15]
a - b  # [-5 -5 -5 -5 -5]
a * b  # [ 6 14 24 36 50]
a / b  # [0.167 0.286 0.375 0.444 0.5]
a // b  # [0 0 0 0 0]
a ** b  # [1 128 6561 262144 9765625]
```

PART 2 • TOPIC 2

NumPy Broadcasting Mechanics

Broadcasting describes how NumPy handles arithmetic between arrays of different shapes. Under specific compatibility rules, the smaller array is virtually 'stretched' across the larger one — with zero extra memory copies.

• • • PYTHON

```
# Python loop (slow, high overhead)
python_list = [1, 2, 3, 4, 5]
for i in range(len(python_list)):
    python_list[i] += 10

# NumPy broadcasting (fast, vectorized)
arr = np.array([1, 2, 3, 4, 5])
arr = arr + 10      # scalar 10 stretched across every element

# Broadcasting across a full 2D grid
grid = np.arange(1, 31).reshape(5, 6)
grid + 10           # 10 stretched across all rows & columns
```

► OUTPUT

```
[11 12 13 14 15]
```

★ KEY INSIGHT

Broadcasting is the single biggest reason NumPy code both reads cleaner and runs dramatically faster than raw Python loops.

PART 2 • TOPIC 3

Memory Management — Views, References & Deep Copies

This is one of the most important — and most bug-prone — concepts in NumPy: knowing whether an operation returns a new array or a shared reference to the original memory block.

• • • PYTHON

```
source = np.arange(1, 20)

# A. Shared reference (dangerous) — direct assignment
view_ref = source
view_ref[0] = 99
# source[0] is ALSO 99 — they point to the same memory block!

# B. Safe deep copy — fully independent
safe_copy = source.copy()
safe_copy[0] = 55
# source is UNTOUCHED — remains 99, not 55
```

★ KEY INSIGHT

Golden rule: use `.copy()` any time you need to modify data without affecting the original array. Plain assignment (`=`) never copies data.

PART 2 • TOPIC 4

Matrix Multiplication & Transposition

NumPy distinguishes sharply between element-wise multiplication and true linear-algebra matrix multiplication (dot product).

• • • PYTHON

```
A = np.arange(1, 5).reshape(2, 2) # [[1 2],[3 4]]
B = np.arange(5, 9).reshape(2, 2) # [[5 6],[7 8]]

# A. Hadamard (element-wise) product
A * B      # [[ 5 12],[21 32]]

# B. True matrix dot product
A @ B      # [[19 22],[43 50]]
np.dot(A, B) # identical result

# C. Transpose — flips rows and columns
A.T        # [[1 3],[2 4]]
```

★ KEY INSIGHT

'*' never performs true matrix multiplication in NumPy — always use '@' or `np.dot()` for linear-algebra products.

PART 2 • TOPIC 5

Structural Combinations — Stacking Mechanics

Multiple arrays can be joined together along different axes using dedicated stacking functions.

• • • PYTHON

```
v1 = np.arange(1, 5) # [1 2 3 4]
v2 = np.arange(5, 9) # [5 6 7 8]

np.vstack((v1, v2))
# stack as rows    → [[1 2 3 4]
#                  [5 6 7 8]]
np.hstack((v1, v2))    # link end-to-end    → [1 2 3 4 5 6 7 8]

np.column_stack((v1, v2)) # pair as columns    → [[1 5]
#                                                [2 6]
#                                                [3 7]
#                                                [4 8]]
```

PART 2 • TOPIC 6

Structural Segregation — Array Splitting Mechanics

Splitting is the mirror opposite of stacking — dividing a single array into segments.

• • • PYTHON

```
m = np.arange(16).reshape(4, 4)

np.hsplit(m, 4) # split into 4 vertical column slices
np.vsplit(m, 2) # split into 2 horizontal row-blocks
```

► OUTPUT

```
hsplit section [0]:
[ [0]
  [4]
  [8]
  [12] ]
... up to section [3]

vsplit section [0]:
[[0 1 2 3]
 [4 5 6 7]]

vsplit section [1]:
[[ 8  9 10 11]
 [12 13 14 15]]
```

PART 03

Advanced Data Cleaning & Logical Pipelines

NaN Handling, Conditional Logic & Sorting

5 Core Topics · Fully Worked Examples & Outputs

PART 3 • TOPIC 1

The Imperfect Data Toolkit — Managing NaN and Inf

Real-world datasets frequently contain broken values: `np.nan` ('Not a Number', a placeholder for missing/corrupt data) and `np.inf` (infinity, e.g. from division by zero).

Standard aggregations like `.sum()` or `.mean()` silently fail and return `nan` if even one NaN is present — 'NaN-safe' functions must be used instead.

• • • PYTHON

```
data = np.array([10.0, 20.0, np.nan, 40.0, np.inf])

np.isnan(data)  # detect NaN positions
np.isinf(data)  # detect Inf positions

clean_mask = ~np.isnan(data) & ~np.isinf(data)
data[clean_mask]  # fully cleaned data

nan_arr = np.array([5, 10, np.nan, 15])
nan_arr.mean()    # nan (fails)
np.nanmean(nan_arr)  # 10.0 (succeeds)
```

★ KEY INSIGHT

Common NaN-safe functions: `np.nanmean`, `np.nansum`, `np.nanmax`, `np.nanmin`, `np.nanstd`.

PART 3 • TOPIC 2

Vectorized Conditionals via `np.where()`

`np.where(condition, value_if_true, value_if_false)` works like an optimized, vectorized ternary / if-else switch applied to an entire array at once.

• • • PYTHON

```
scores = np.array([45, 82, 61, 30, 95, 74])

grades = np.where(scores >= 60, 'PASS', 'FAIL')
# ['FAIL' 'PASS' 'PASS' 'FAIL' 'PASS' 'PASS']

# Keep value if condition true, else floor at 50
clamped = np.where(scores >= 50, scores, 50)
# [50 82 61 50 95 74]
```

► OUTPUT

```
['FAIL' 'PASS' 'PASS' 'FAIL' 'PASS' 'PASS']
[50 82 61 50 95 74]
```

PART 3 • TOPIC 3

Advanced Multi-Criteria Conditional Masking

Standard Python keywords (and, or, not) cannot be used on full NumPy arrays — they evaluate truthiness for the whole object at once. Use the bitwise vectorized operators instead: **&** (AND), **|** (OR), **~** (NOT).

• • • PYTHON

```
ages = np.array([12, 25, 42, 19, 65, 31, 8])

adult_mask = (ages >= 18) & (ages < 60)
ages[adult_mask] # [25 42 19 31]
```

► OUTPUT

```
[25 42 19 31]
```

★ KEY INSIGHT

Always wrap each individual condition in parentheses when combining with **&** or **|** — operator precedence rules require it.

PART 3 • TOPIC 4

Global Identification & Truth-Value Pipelines

NumPy provides fast global inspection tools: **np.unique** for distinct values and frequency counts, and **np.any** / **np.all** for truth-value evaluation.

• • • PYTHON

```
trials = np.array([5, 5, 2, 8, 2, 9, 5, 12, 8])

uniques, counts = np.unique(trials, return_counts=True)
# uniques: [ 2  5  8  9 12] counts: [2 3 2 1 1]

flags = np.array([False, True, False, False])
np.any(flags) # True — at least one True
np.all(flags) # False — not all True
```

PART 3 • TOPIC 5

Data Reordering — Sort vs. Argsort

`np.sort()` returns a new sorted copy of the values themselves. `np.argsort()` is subtly different — it returns the index positions that would produce sorted order, which is essential for keeping multiple aligned datasets in sync.

• • • PYTHON

```
arr = np.array([80, 20, 50, 10, 90])

np.sort(arr)    # [10 20 50 80 90]

idx = np.argsort(arr)
# idx = [3 1 2 0 4] (index 3 holds the smallest value 10, etc.)

arr[idx]        # reconstructs the sorted order: [10 20 50 80 90]
```

★ KEY INSIGHT

`argsort()` is especially powerful when you need to reorder a second, 'parallel' array using the sort order derived from a first array.

PART 04

Advanced Mathematical Tools & Benchmarking

Universal Functions, Linear Algebra & Performance Proof

4 Core Topics · Fully Worked Examples & Outputs

PART 4 • TOPIC 1

Universal Functions (ufuncs) for Scaled Trig & Math

NumPy 'ufuncs' are highly optimized mathematical functions that execute directly on underlying C arrays and automatically apply element-wise across an entire array in a single vectorized call.

• • • PYTHON

```
angles = np.array([0, np.pi/4, np.pi/2, np.pi])

np.sin(angles)
np.cos(angles)

vals = np.array([1.0, 2.71828, 10.0, 100.0])
np.log(vals)    # natural logarithm
np.log10(vals)  # base-10 logarithm
np.sqrt(vals)   # square root
```

PART 4 • TOPIC 2

Elementary Linear Algebra Engine (np.linalg)

The `np.linalg` submodule solves systems of simultaneous equations, computes determinants, and calculates matrix inverses — core operations behind regression and optimization algorithms.

• • • PYTHON

```
# Solve: 2x + 3y = 8
#       x + 2y = 5
coeffs = np.array([[2, 3], [1, 2]])
constants = np.array([8, 5])

np.linalg.inv(coeffs)          # matrix inverse
np.linalg.solve(coeffs, constants)  # x: 1.0, y: 2.0
```

► OUTPUT

```
x = 1.0, y = 2.0
```

PART 4 • TOPIC 3

Collapsing Dimensions — Flatten vs. Ravel

Both methods compress a multi-dimensional grid into a single 1D vector, but they manage memory very differently.

.flatten()	.ravel()
◆ Creates a deep, independent physical copy of the data. ◆ Safe — modifying the result never touches the source. ◆ Uses more memory (full duplication).	◆ Creates a shared reference / view pointer wherever possible. ◆ Faster and uses minimal extra memory. ◆ Modifying the result CAN alter the original source array.

```
• • • PYTHON

grid = np.array([[1, 2], [3, 4]])

flat = grid.flatten()
flat[0] = 999    # grid stays untouched: [[1 2],[3 4]]

rav = grid.ravel()
rav[0] = 999    # grid IS altered: [[999 2],[3 4]]
```

PART 4 • TOPIC 4

Performance Benchmarking — Python Loops vs. NumPy Vectors

The single most convincing argument for NumPy is raw performance. Adding one million elements pairwise with a plain Python loop versus a vectorized NumPy operation reveals a dramatic gap in both memory and speed.

```
• • • PYTHON

N = 1_000_000
py_x, py_y = list(range(N)), list(range(N))
np_x, np_y = np.arange(N), np.arange(N)

# Python loop
result = [x + y for x, y in zip(py_x, py_y)]

# NumPy vectorized
result = np_x + np_y
```

► OUTPUT

Memory (Python List): ~35+ MB

Memory (NumPy Array): ~8 MB

Time (Python Loop): ~0.07 - 0.15 s

Time (NumPy Vector): ~0.001 - 0.003 s

→ NumPy is typically 30x - 100x faster on large-scale operations.

★ KEY INSIGHT

Exact numbers vary by machine, but the pattern is universal: vectorized NumPy code scales far better than pure Python loops.

PART 05

Structural Data Mutations & Vector Expansion

Type Casting, Dimensional Control & Value Clamping

5 Core Topics · Fully Worked Examples & Outputs

PART 5 • TOPIC 1

Precision Control — Explicit Type Casting (.astype)

While NumPy manages upcasting automatically, you often need to manually demote or change precision explicitly — for example, to save GPU memory or match a specific machine-learning tensor dtype.

• • • PYTHON

```
vals = np.array([1.9, 2.4, 5.8, 9.1])
vals.dtype      # float64

cast = vals.astype(np.int32)
cast           # [1 2 5 9] — truncated, NOT rounded
cast.dtype     # int32
```

► OUTPUT

```
[1 2 5 9]
```

★ KEY INSIGHT

`astype()` truncates decimals toward zero rather than rounding — always verify this behavior matches your intent.

PART 5 • TOPIC 2

Dimensional Inflation — `np.newaxis` & `np.expand_dims`

Deep learning frameworks often expect an explicit batch dimension. You can transform a flat 1D vector into a 2D row or column matrix without copying or overwriting any data.

• • • PYTHON

```
v = np.array([10, 20, 30])    # shape: (3,)

v[np.newaxis, :]             # row matrix → shape (1, 3)
v[:, np.newaxis]             # column matrix → shape (3, 1)
np.expand_dims(v, axis=0)     # same as row matrix → (1, 3)
```

PART 5 • TOPIC 3

Dimensional Stripping — Compressing Axes (.squeeze)

The opposite of expansion: `.squeeze()` deletes every dimension whose size equals 1, cleanly flattening down unnecessary processing channels.

• • • PYTHON

```
t = np.array([[[10], [20], [30]]]) # shape (1, 3, 1)

t.squeeze()      # [10 20 30]
t.squeeze().shape # (3,)
```

► OUTPUT

```
[10 20 30]
```

PART 5 • TOPIC 4

Multi-Dimensional Coordinate Extraction (`np.argwhere`)

While `.argmax()` works on a flattened index, `np.argwhere()` returns the exact Row/Column coordinate pairs that satisfy a condition.

• • • PYTHON

```
grid = np.array([[0, 15, 0], [0, 0, 42], [8, 0, 0]])

np.argwhere(grid > 0)
```

► OUTPUT

```
[[0 1] -> value 15
 [1 2] -> value 42
 [2 0]] -> value 8
```

PART 5 • TOPIC 5

Data Stabilization & Value Clamping (`np.clip`)

`np.clip()` forces outlier data into a strict minimum/maximum boundary — any value below the minimum becomes the minimum, any value above the maximum becomes the maximum. This is a standard technique for preventing exploding values in ML pipelines.

• • • PYTHON

```
signals = np.array([-15, 4, 12, 98, 35, -2, 50])

np.clip(signals, a_min=0, a_max=40)
```

► OUTPUT

```
[ 0  4 12 40 35  0 40]
```

★ KEY INSIGHT

`np.clip` is commonly used to prevent 'exploding gradients' during neural network training by bounding extreme values.